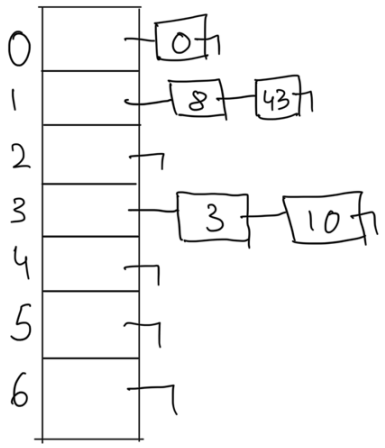
CS2210 Data Structures and Algorithms – Second Concept Assignment

Name: Dhruwang Kishorbhai Akbari Student No. 251186352

Q1.




Q2.

| | |
|---|---|
| 0 | 0 |
| 1 | 8 |
| 2 | 43 |
| 3 | 3 |
| 4 | 10 |
| 5 | |
| 6 | |

Q3.

| | |
|---|---|
| 0 | 43 |
| 1 | 10 |
| 2 | |
| 3 | 3 |
| 4 | |
| 5 | 8 |
| 6 | 0 |

Q4. Solving the recurrence equation

$$f(0) = c_0$$

$$f(n) = f(n-2) + c_1 n + c_2$$

$1. f(n) = f(n-2) + n$ (we can remove the constants from the equation so we are left with n

$f(n-2) = f(n-4) + (n-2)$ now we substitute this equation to 1

$$2. f(n) = \big(f(n-4) + (n-2)\big) + n$$

$f(n-4) = f(n-6) + (n-4)$ Now we compute and we substitute in equation 1

$$3. f(n) = (f(n-6) + (n-4)) + (n-2) + n$$

Now we repeat these steps of substituting for 'k' times

$$4. f(n) = f(n-k) + \big(n-(k-2)\big) + \big(n-(k-4)\big) + \cdots + (n-1) + n$$

Now that we have reached n = k, we substitute the value of k in equation 4

$$5. f(n) = f(n-n) + (n-n+2) + (n-n+4) + \cdots + (n-1) + n$$

$$6. f(n) = f(0) + 2 + 4 + \cdots + (n-1) + n$$

$$7. f(n) = c_0 + \sum_{i=2}^{n} i$$

When we solve the summation we get $n(n+1)$

$$8. f(n) = c_0 + n(n+1)$$

Now removing the constants, we get the time complexity as $O(n^2)$.

Q5.

Part i) Odd.java

Part ii)

The worst case time complexity would be when there are no odd degree nodes. This could be the case with Proper Binary Trees where there is always even degree of nodes, or any other trees with even degree of nodes. This means that the algorithm will traverse through the whole tree. For any type of traversing the tree we usually do pre-order, post-order or level order. We know that for traversing in the trees we usually have a recursive algorithm. For every node in the tree there is one recursive call made and for every call we do constant operations on the node (visiting the node). Hence, the time complexity of the traversal would O(n).

Q6. We know the size of the array is going to be $s = \frac{n(n+1)}{2}$ and the while loop will run until the value of $j < s$. Now, the value of $i$, is being increased by 1, and the value of $j$, is being increased in accordance with the value of $i$. So if we choose the value of $n = 3$, the size of the array is 6. In the 1st iteration of the loop, $i = 1, j = 1$. In 2nd iteration of loop, $i = 2, j = 3$.In the 3rd iteration of the loop, $i = 3, j = 6$. The loop won't run 4th time as $6 < 6$ is false. This means that the loop ran 3 times and which was the value of n. We know that the loop runs n times and does constant operations of assigning the value in the array and increasing the value of i and j. So we come to know the time complexity of the algorithm to be O(n).